

Reviewer Pack — Ihara Zeta Entropy Predicts Tseitin DPLL Tree Size

Entry 3ba5471eff37 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 07:23:23 UTC

Ihara Zeta Entropy Predicts Tseitin DPLL Tree Size

Entry ID: 3ba5471eff37 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 07:23:23 UTC

1. Conjecture

Field A (mathematical branch): Ihara zeta function of finite graphs (Hashimoto–Stark–Terras combinatorial graph-zeta theory rooted in number-theoretic L-functions; essentially no papers connect Ihara zeta values to proof complexity, distinct from non-backtracking spectral approaches) **Field B** (complexity object): Tree-like Resolution / DPLL refutation tree size for Tseitin XOR formulas $T(G, \omega)$ on 3-regular graphs (Urquhart–Ben-Sasson–Razborov–Atserias regime)

Statement:

Let G be a connected 3-regular graph on n vertices with adjacency matrix A , and let $\omega: V \rightarrow \{0,1\}$ have $\sum \omega$ odd, so $T(G, \omega)$ is unsatisfiable. Define the Ihara entropy $H(G) := (1/n) \cdot \ln \det((11/9) \cdot I - A/3)$, which is well-defined and positive because $(11/3) \cdot I - A$ has eigenvalues in $[2/3, 20/3]$ for any 3-regular G . We conjecture that with pre-registered absolute constants $c_1 = 1/20$ and $c_2 = 20$, every connected 3-regular G with odd ω satisfies $c_1 \cdot n \cdot H(G) - 1 \leq \log_2(S_DPLL(T(G, \omega))) \leq c_2 \cdot n \cdot H(G) + \log_2(n) + 1$, where S_DPLL is the leaf count of the canonical DPLL refutation tree using unit propagation plus first-unassigned-variable branching. A single (G, ω) violating either inequality refutes the conjecture.

Rationale (proposer’s reasoning):

The Ihara zeta function $\zeta_G(u) = \prod_p (1 - u^{|p|})^{-1}$ over prime non-backtracking cycles is encoded by $\det(I - uA + u^2(D - I))$; at $u = 1/3$ this determinant aggregates how cycles of all lengths contribute multiplicatively, and each cycle of G corresponds in $T(G, \omega)$ to a closed XOR constraint that DPLL must locally certify, so per-vertex cycle entropy should track per-vertex branching entropy. The evaluation point $u = 1/3$ sits strictly inside the radius of convergence for 3-regular graphs, giving a numerically stable, integer-matrix invariant whose growth on Ramanujan-type expanders matches Urquhart’s exponential refutation-size lower bound. This bridges number-theoretic graph zetas to proof complexity in a regime where, to our knowledge, fewer than five papers have looked.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): db01b27fb056c3a6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 instances ($30 \text{ seeds} \times n \in \{6,8,10,12,14\}$), every instance must satisfy $(1/20) \cdot n \cdot H(G) - 1 \leq \log_2(S_DPLL) \leq 20 \cdot n \cdot H(G) + \log_2(n) + 1$, AND the Spearman $\rho(\log_2 S, n \cdot H(G))$ over all 150 instances must exceed 0.6. Any single violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Ihara zeta function Tseitin formula resolution proof complexity - non-backtracking spectrum 3-regular graph DPLL tree-like resolution lower bound - graph zeta function expander Tseitin XOR refutation size

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def matrix_mult(a, b):
    n = len(a)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += a[i][k] * b[k][j]
    return result

def matrix_sub(a, b):
    n = len(a)
    return [[a[i][j] - b[i][j] for j in range(n)] for i in range(n)]

def matrix_scale(a, scalar):
    n = len(a)
    return [[a[i][j] * scalar for j in range(n)] for i in range(n)]

def matrix_det(matrix):
```

```

n = len(matrix)
if n == 1:
    return matrix[0][0]
det = 0
for col in range(n):
    minor = [row[:col] + row[col+1:] for row in matrix[1:]]
    det += ((-1) ** col) * matrix[0][col] * matrix_det(minor)
return det

def generate_3_regular_graph(n):
    if n % 2 != 0:
        raise ValueError("n must be even for a 3-regular graph")
    vertices = list(range(n))
    adj = defaultdict(list)
    remaining_vertices = vertices.copy()
    random.shuffle(remaining_vertices)
    while remaining_vertices:
        v1 = remaining_vertices.pop()
        for _ in range(3):
            if not remaining_vertices:
                break
            v2 = random.choice(remaining_vertices)
            adj[v1].append(v2)
            adj[v2].append(v1)
            remaining_vertices.remove(v2)
    return adj

def generate_random_3_regular_graph(n):
    while True:
        adj = generate_3_regular_graph(n)
        if len(adj) == n and all(len(adj[v]) == 3 for v in adj):
            return adj

def is_connected(adj):
    if not adj:
        return False
    visited = set()
    stack = [next(iter(adj))]
    while stack:
        v = stack.pop()
        if v not in visited:
            visited.add(v)
            stack.extend(adj[v])
    return len(visited) == len(adj)

def generate_connected_3_regular_graph(n):
    while True:
        adj = generate_random_3_regular_graph(n)
        if is_connected(adj):
            return adj

def generate_omega(n):
    omega = [random.randint(0, 1) for _ in range(n)]
    if sum(omega) % 2 == 0:
        omega[0] = 1 - omega[0]
    return omega

```

```

def build_adjacency_matrix(adj, n):
    matrix = [[0] * n for _ in range(n)]
    for v in adj:
        for u in adj[v]:
            matrix[v][u] = 1
    return matrix

def compute_ihara_entropy(adj, n):
    A = build_adjacency_matrix(adj, n)
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    term1 = matrix_scale(I, 11/9)
    term2 = matrix_scale(A, 1/3)
    matrix = matrix_sub(term1, term2)
    det = matrix_det(matrix)
    if det <= 0:
        return float('nan')
    return (1/n) * math.log(det)

def dpll(clauses, assignment, n):
    if all(any(literal in assignment for literal in clause) for clause in clauses):
        return 1
    unit_clause = next((clause for clause in clauses if len(clause) == 1 and not any(literal in as
    if unit_clause:
        literal = unit_clause[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        return dpll(clauses, new_assignment, n)
    for var in range(1, n+1):
        if var not in assignment and -var not in assignment:
            new_assignment = assignment.copy()
            new_assignment[var] = True
            result = dpll(clauses, new_assignment, n)
            new_assignment[var] = False
            result += dpll(clauses, new_assignment, n)
        return result
    return 0

def generate_tseitin_cnf(adj, omega, n):
    clauses = []
    for v in adj:
        neighbors = adj[v]
        for i in range(3):
            for j in range(i+1, 3):
                u, w = neighbors[i], neighbors[j]
                clauses.append([-v, -u, -w])
                clauses.append([v, u, w])
                clauses.append([-v, u, -w])
                clauses.append([v, -u, w])
    for v in range(n):
        clauses.append([-v if omega[v] == 0 else v])
    return clauses

def run_trial(seed):
    random.seed(seed)
    n_values = [6, 8, 10, 12, 14]

```

```

metric_values = []
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    adj = generate_connected_3_regular_graph(n)
    omega = generate_omega(n)
    A = build_adjacency_matrix(adj, n)
    h = compute_ihara_entropy(adj, n)
    if math.isnan(h):
        continue
    clauses = generate_tseitin_cnf(adj, omega, n)
    leaves = dpll(clauses, {}, n)
    log_leaves = math.log2(leaves) if leaves > 0 else 0
    lower_bound = (1/20) * n * h - 1
    upper_bound = 20 * n * h + math.log2(n) + 1
    if log_leaves < lower_bound or log_leaves > upper_bound:
        conjecture_holds = False
        counterexample = f"n={n}, seed={seed}, log_leaves={log_leaves}, lower_bound={lower_bound}"
        break
    metric_values.append(log_leaves)
    instances_tested += 1

return {
    "metric_name": "log2(DPLL leaves)",
    "metric_value": sum(metric_values) / len(metric_values) if metric_values else 0,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    mean = sum(metric_values) / len(metric_values) if metric_values else 0
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values)) if metric_values else 0
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results) if results else 0

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode=124) without producing a RESULT line or any instance data, so neither the bound inequalities nor the Spearman ρ threshold could be evaluated. | next: Reduce the instance budget (e.g. fewer seeds or cap n at 12) and/or replace the exact DPLL leaf count with an iterative bounded search so the 150-instance sweep completes within the timeout.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	173706
2	preregistration	claude_max	opus	0	0	6195
3	novelty	claude_max	opus	0	0	3261
4	novelty	claude_max	opus	0	0	7473
5	test_gen	mistral	codestral-latest	0	0	316822
6	test_gen	mistral	codestral-latest	0	0	315934
7	test_gen	mistral	codestral-latest	0	0	316975
8	test_gen	mistral	codestral-latest	0	0	314046
9	judge	claude_max	opus	0	0	4540

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1458952 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in *research/audit/3ba5471eff37.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/3ba5471eff37.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/3ba5471eff37.tar.gz* (if generated)